feasibility

Deep Research — Android emulator under rootless Podman on macOS arm64 (HXC-108 Android gap)

Date: 2026-06-23 · **Host:** macOS 15.5 (Darwin 24.5.0), Apple M3 Pro (T6030), arm64
Mandate: §11.4.150 (mandatory multi-angle deep research before a structural verdict), §11.4.8 / §11.4.99 (latest authoritative sources), §11.4.112 (structural-impossibility classification). **Context:** The §6.X guard blocks host-direct `emulator`/`adb` as gate evidence. The §6.X-sanctioned runner `helix_code/scripts/run-challenge-matrix.sh` does not exist. Question: can it drive an Android emulator inside a **rootless Podman container on this macOS host** to produce a §11.4.158 on-device recording?

VERDICT — §11.4.112 STRUCTURALLY-IMPOSSIBLE on this host (accelerated, gate-eligible path)

Running an Android emulator inside a rootless Podman Linux container on Apple Silicon macOS, as a hardware-accelerated **gate-eligible** runner, is **structurally impossible**. Apple's `Hypervisor.framework` (HVF) — the only macOS acceleration backend — is a macOS-**host-only** API. A Podman container on macOS runs inside a Linux VM managed by the closed-source `Virtualization.framework`/`applehv` provider, which **does not expose `/dev/kvm`** to the guest or its containers. The Android emulator inside that container therefore has no accelerator and falls back to unusably-slow software emulation (or fails). macOS 15 Sequoia added nested-virt for M3+, but that is **not plumbed through podman-machine's `applehv` path to surface `/dev/kvm` in containers** — relying on it would be a guess, not a finding (§11.4.6). This is a closed-platform-API constraint, not missing engineering.

The Containers submodule already encodes this verdict as FACT — see Angle 4.

Evidence (per angle; FACT unless marked UNCONFIRMED)

Angle 1 — Podman `applehv` on Apple Silicon & `/dev/kvm` (FACT). On macOS, `podman machine` runs a Linux VM via the native `applehv` (`Virtualization.framework`) provider, which is closed-source and "can't be extended to implement new emulated devices to be exposed to guests" — so exposing `/dev/kvm` to nested containers is a known architectural limitation of the applehv path. - https://www.redhat.com/en/blog/podman-mac-machine-architecture - https://sinrega.org/2024-03-06-enabling-containers-gpu-macos/

Angle 2 — Android emulator in Docker/Podman precedents (FACT). Google's `android-emulator-container-scripts` require `/dev/kvm` ("KVM must be available… bare metal or a (V)M that provides nested virtualization"). The only KVM-free containerized-Android path is **Redroid**, which runs the Android *userspace* on the **host Linux kernel** via binder/ashmem — unavailable on macOS (no Linux host kernel). Every containerized-emulator path needs host KVM or a host Linux kernel; macOS gives a container neither. - https://github.com/google/android-emulator-container-scripts - https://github.com/google/android-emulator-container-scripts/issues/21 - https://codersera.com/blog/android-emulator-docker-without-kvm/

Angle 3 — Apple Silicon nested virtualization (FACT + honest nuance). macOS 15 Sequoia added hardware-assisted nested virtualization for M3+ (this host IS M3 Pro), theoretically allowing KVM inside a Linux VM. **UNCONFIRMED (no authoritative source for):** that podman-machine's `applehv` plumbs this through to expose `/dev/kvm` inside containers. The closed-source device-exposure limit (Angle 1) blocks it. The hardware capability exists but is not surfaced through the sanctioned podman path. - https://news.ycombinator.com/item?id=40642328 - https://forum.parallels.com/threads/macos-15-sequoia-nested-virtualization-for-m3-macs.364397/

Angle 4 — Containers submodule (FACT, in-repo — strongest evidence). `submodules/containers/pkg/emulator/accel.go` states verbatim: *"macOS hosts have no `/dev/kvm`… A Linux container running under podman/docker on macOS executes inside a Linux VM and cannot reach the host's HVF interface… on macOS the host-direct runner is the only accelerated path AND the gate-eligible runner."* `pkg/cuttlefish/accel.go` independently encodes the same: Cuttlefish requires `/dev/kvm`, absent on macOS → `RequireKVM()` errors → SKIP-with-reason per §11.4.3, never PASS. The submodule already provides `AccelProfileForOS`, `ResolveRunner("auto")`, `GateEligibleForOS`, `cmd/emulator-matrix`, and `cmd/emulator-canary`. The `containerized.go --device /dev/kvm` path is gate-eligible **only on a Linux x86_64 host with KVM**.

Angle 5 — Community reports (FACT). No credible report of an accelerated Android emulator inside a Podman container on macOS Apple Silicon. The documented alternative is host-direct (native macOS emulator using HVF).

Containers-submodule gap

No gap to fill for the macOS-rootless-podman path — it is provably non-viable and the submodule already documents it as such. Authoring `run-challenge-matrix.sh` to drive an emulator through a rootless-podman container on macOS would require extending the submodule to do the structurally-impossible, which §11.4.74 / §11.4.112 forbid. No upstream extension is warranted for this path.

Recommendation

Classify *"Android emulator inside rootless podman on macOS arm64"* as **§11.4.112 `Won't-fix: structurally-impossible`**, cited constraint: *Apple* `Virtualization.framework/applehv` *is closed-source and does not expose* `/dev/kvm` *to guest containers; HVF is a host-only API unreachable from a Linux container* (already FACT-encoded in `pkg/emulator/accel.go` + `pkg/cuttlefish/accel.go`). Do NOT re-attempt without NEW evidence the platform changed (§11.4.7 / §11.4.34).

Honest alternatives for sanctioned §6.X on-device Android recording (operator decision per §11.4.66 / §11.4.122 / §11.4.143 / §11.4.158): 1. **Linux x86_64 KVM gate host (canonical).** `run-challenge-matrix.sh --runner containerized` on a Linux x86_64 box with `/dev/kvm` IS the gate-eligible accelerated container path — needs no new submodule code, just a gate host (local / CI / a `CONTAINERS_REMOTE_HOST_*` enrolment). 2. **macOS host-direct, reclassified.** `host-direct` is gate-eligible per the submodule's accel model on macOS, but §6.X blocks it. Requires an operator decision: carve a sanctioned macOS host-direct exception, OR treat macOS Android recording as `operator_attended` SKIP (§11.4.52) + migration item. 3. **Physical Android device (operator-attended).** Real device over USB/ADB, recorded — the §11.4.143 real-journey path, honestly operator-attended.

Net: route §6.X Android-client gate recording to a Linux x86_64 KVM host via the existing `--runner containerized` path. The rootless-podman-on-macOS approach is structurally impossible and should be closed, not engineered.

Sources verified 2026-06-23

- https://www.redhat.com/en/blog/podman-mac-machine-architecture

- https://sinrega.org/2024-03-06-enabling-containers-gpu-macos/
- https://github.com/google/android-emulator-container-scripts
- https://github.com/google/android-emulator-container-scripts/issues/21
- https://codersera.com/blog/android-emulator-docker-without-kvm/
- https://news.ycombinator.com/item?id=40642328
- https://forum.parallels.com/threads/macos-15-sequoia-nested-virtualization-for-m3-macs.364397/
- in-repo: `submodules/containers/pkg/emulator/accel.go`, `pkg/cuttlefish/accel.go`, `README.md`